OPERATING SYSTEMS

ORANGE-PROBLEM-2 REPORT

Name : Dikshith Varma M N

Class : AIML-G

SRN : PES1UG24AM363

ABSTRACT:

This project implements a simplified version control system (PES-VCS) inspired by Git. It supports file tracking, staging, commit creation, and history traversal using content-addressable storage. The system uses SHA-256 hashing to ensure data integrity and efficient storage. The project demonstrates core operating system concepts such as file management, indexing, and persistent storage.

1.AIM :

To design and implement a simplified version control system (PES-VCS) that supports file tracking, staging, commit creation, and history traversal using content-addressable storage.

IMPLEMENTATION

PHASE 1: Object Storage foundation

Screenshot 1A: Output of ./test_objects showing all tests passing.

```
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: find .pes/objects -type f showing the sharded directory structure.

```
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$
```

PHASE 2: Tree Objects

Screenshot 2A: Output of ./test_tree showing all tests passing.

```
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$
```

Phase 3: The Index (Staging Area)

Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ ./pes add file1.txt file2.txt
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  modified:   file1.txt
  modified:   file2.txt

Untracked files:
  untracked:  tree.c
  untracked:  tree.h
  untracked:  pes.c
  untracked:  commit.c
  untracked:  .gitignore
  untracked:  test_tree.c
  untracked:  Makefile
  untracked:  commit.h
  untracked:  index.c
  untracked:  README.md
  untracked:  test_objects
  untracked:  index.h
  untracked:  test_tree
  untracked:  test_sequence.sh
  untracked:  pes.h
  untracked:  object.c
  untracked:  test_objects.c
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776353042 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776353042 6 file2.txt
```

Phase 4: Commits and History

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: be64e4a00cb0... Initial commit
Committed: 742895f35bdd... Add world
Committed: 6554d7ebce6c... Add farewell
commit 6554d7ebce6c362f995f36c204f8edb844709f3fc9c1909d60fb9d035c92784b
Author: Dikshith Varma <PES1UG24AM363>
Date:   1776354431

    Add farewell
er auth!

commit 742895f35bdd96431291bd33b7ba534df7f676ac82b1477f5f72d69e8c3bdbe8
Author: Dikshith Varma <PES1UG24AM363>
Date:   1776354431

    Add world
ll
er auth!

commit be64e4a00cb0f392b18b4dadace54ad63f21843b2c643f87d82d5907eaca521f
Author: Dikshith Varma <PES1UG24AM363>
Date:   1776354431
```

```
    Initial commit
 auth!

commit 8195517872fb8dab54d15caaf2436bc7e9efe1c647e817e9c6fa544a47a37ffe
Author: PES User <pes@localhost>
Date:   1776354180

    Add farewell
d f◆

commit c28922cb82c98b187dd05d6547c366fee7c6ceb3519408cce6f5b5f275157302
Author: PES User <pes@localhost>
Date:   1776354180

    Add world
ll
d f◆

commit 8b2d076ee5f243697d63a8a0c3b21711eb27e9337db1266daa86dea35dcea6ef
Author: PES User <pes@localhost>
Date:   1776354180

    Initial commit
f◆

commit 22fe02564a3a215681384dc0832321481ef31e4dd9a5cbac52d490b6d97ca960
Author: PES User <pes@localhost>
Date:   1776354095

    Initial commit
ial commiQ
```

Screenshot 4B: find .pes -type f | sort showing object store growth after three commits.

```
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/22/fe02564a3a215681384dc0832321481ef31e4dd9a5cbac52d490b6d97ca960
.pes/objects/3b/c19934c2ccc37b9582dd42105ca4a1df8e197f51c99a3ef71628231437fdbf
.pes/objects/65/54d7ebce6c362f995f36c204f8edb844709f3fc9c1909d60fb9d035c92784b
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/74/2895f35bdd96431291bd33b7ba534df7f676ac82b1477f5f72d69e8c3bdbe8
.pes/objects/81/95517872fb8dab54d15caaf2436bc7e9efe1c647e817e9c6fa544a47a37ffe
.pes/objects/8b/2d076ee5f243697d63a8a0c3b21711eb27e9337db1266daa86dea35dcea6ef
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/be/64e4a00cb0f392b18b4dadace54ad63f21843b2c643f87d82d5907eaca521f
.pes/objects/c2/8922cb82c98b187dd05d6547c366fee7c6ceb3519408cce6f5b5f275157302
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$
```

Screenshot4C: cat .pes/refs/heads/main and cat .pes/HEAD s howing the reference chain.

```
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$ cat .pes/refs/heads/main
6554d7ebce6c362f995f36c204f8edb844709f3fc9c1909d60fb9d035c92784b
dikshith@legacy:~/Desktop/PES1UG24AM363-pes-vcs$
```

Phase 5 & 6: Analysis-Only Questions

The following questions cover filesystem concepts beyond the implementation scope of this lab. Answer them in writing — no code required.

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Solution:

A branch in PES is stored as a file inside .pes/refs/heads/ which contains a commit hash.

To implement checkout, first update the .pes/HEAD file so that it points to the selected branch. Then read the commit hash from the branch file.

Next, update the working directory. This means reading the commit object, getting its tree, and then recreating the files and folders accordingly. Files that exist in the new branch should be created, files that don't exist should be removed, and existing files should be replaced with correct content.

This operation is complex because it must not overwrite uncommitted changes, it must correctly handle directories and nested files, and it must keep everything (index, working directory, and object store) consistent.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Solution:

Before switching branches, we must check if the user has modified any files.

For each file in the index, compare the stored hash with the hash of the current file in the working directory. If they are different, the file has been modified.

Then compare the file between the current branch and the target branch. If the file is both modified locally and different between branches, then checkout must be stopped to avoid losing changes.

In simple terms, we compare the working directory, index, and branch versions using hashes to detect conflicts.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Solution:

Detached HEAD means that HEAD is pointing directly to a commit instead of a branch.

In this state, you can still make commits, but those commits are not connected to any branch. This means they are not permanently saved and can be lost later.

To recover such commits, you can use the log to find the commit hash and then create a new branch pointing to it.

So, detached HEAD allows commits, but they must be saved by creating a branch.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Solution:

Over time, some objects in the repository are no longer used by any branch.

To remove them, start from all branch heads and traverse all commits, trees, and blobs. Mark all objects that are reachable. Then, delete any object that is not marked.

A hash set can be used to store reachable objects because it allows fast lookup.

For a repository with 100,000 commits and 50 branches, many commits are shared, so roughly 300,000 to 500,000 objects may need to be visited.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Solution:

Running garbage collection during a commit can cause problems.

While a commit is in progress, new objects are created but not yet linked to any branch. If garbage collection runs at that moment, it may think these objects are unused and delete them.

After that, the commit will reference objects that no longer exist, causing repository corruption.

Git avoids this by using locking, updating references only after objects are fully written, and protecting new objects temporarily.

So, garbage collection must not run while a commit is happening.